PHASE 2: INFERENCE / APPLICATION OF FACE MASK DETECTION MODEL

1 IMPORT REQUIRED LIBRARIES
Import: TensorFlow/Keras, OpenCV, NumPy, Imutils, VideoStream, OS, Time, Argparse
Purpose: Provides functions for image preprocessing, model loading, face detection, camera access, and displaying results.

IMUTILS

2 DEFINE FACE DETECTION AND MASK PREDICTION FUNCTION
Define detect_and_predict_mask().
Input: Frame, faceNet, maskNet
Output: locs (locations), preds (predictions)

3 READ INPUT FRAME DIMENSIONS
Get height (h) and width (w) of the frame.
Needed to calculate face bounding-box coordinates.

h
w

4 CONVERT FRAME INTO BLOB
Use cv2.dnn.blobFromImage():
- Resize to 300×300
- Apply mean subtraction

Purpose: Convert image to the format required by the SSD face detection model.

BLOB

5 FACE DETECTION USING SSD MODEL
Send the blob to the pretrained OpenCV SSD face detector.

Blob → Face Detection Network → Face Detections

6 INITIALIZE STORAGE LISTS
Create empty lists:
- **faces** → stores extracted face images
- **locs** → stores face bounding-box locations
- **preds** → stores mask predictions

faces	[ ]
locs	[ ]
preds	[ ]

7 LOOP THROUGH ALL FACE DETECTIONS & CONFIDENCE CHECK
For each detected face, check confidence.
Is confidence > 0.5?

NO → Ignore (weak detection)
YES → Continue processing

8 EXTRACT FACE BOUNDING BOX
Calculate startX, startY, endX, endY (coordinates of detected face).

9 VALIDATE BOUNDING-BOX COORDINATES
Ensure coordinates are inside frame boundaries.

10 EXTRACT FACE ROI (REGION OF INTEREST)
Crop the face from the original frame using bounding-box coordinates.

11 PREPROCESS FACE IMAGE (for MobileNetV2)
1. Convert BGR → RGB
2. Resize to 224×224
3. Convert image to array
4. Apply preprocess_input (MobileNetV2)

12 STORE FACE AND LOCATION
- Processed face → faces
- Bounding-box coordinates → locs

(Used for prediction and for drawing results later)

13 CHECK WHETHER FACES WERE DETECTED
Any valid faces?

NO → Return empty predictions & go to next frame
YES → Continue to mask prediction

14 CONVERT FACES INTO NUMPY ARRAY
Convert all extracted faces into a NumPy array for batch processing.

NumPy

15 MASK / NO-MASK PREDICTION AND RETURN
Send faces to trained model: FACEMASK_Detector.h5
Model outputs: [Mask Probability, No Mask Probability]
Return: locs (face locations), **preds** (mask predictions)

16 ARGUMENT PARSER
-d → face detector path
-m → mask model path
-c → confidence
Stores values in args dictionary.

17 LOAD FACE DETECTOR
Loads pretrained face detector deploy.prototxt + .caffemodel
↓
faceNet

18 LOAD MASK MODEL
Loads trained face-mask classification model (FACEMASK_Detector.h5)
↓
maskNet

19 START WEBCAM
Use default webcam.
Wait 2 seconds for camera warm-up.

INFINITE VIDEO LOOP
(Keep processing frames continuously)

20 INFINITE VIDEO LOOP
Continuously:
Capture → Detect → Predict → Display → Next Frame → ...

21 CAPTURE FRAME
Capture one frame from webcam.
Resize frame to max width of 400 pixels (faster processing).

22 DETECT FACE + PREDICT MASK
Call detect_and_predict_mask(frame, faceNet, maskNet)
This performs: Frame → Blob → Face Detection → Face ROI → 224×224 → Preprocessing → Mask Model → Prediction
Output: locs, preds

23 EXTRACT COORDINATES
Get the face's bounding box coordinates (startX, startY, endX, endY).

24 EXTRACT MASK PROBABILITIES
Predicts: (mask, withoutMask) = pred

25 DECIDE MASK OR NO MASK
If mask → withoutMask → MASK
Else → NO MASK

26 SELECT COLOR (BGR FORMAT)
Mask → Green (0, 255, 0)
No Mask → Red (0, 0, 255)

27 DISPLAY BOUNDING BOX AND LABEL
If Mask: "Mask: You are allowed" (Green box)
If No Mask: "No Mask: You are not allowed" (Red box)

Mask: You are allowed
No Mask: You are not allowed

28 SHOW WEBCAM OUTPUT
Use cv2.imshow() to display frame with:
- Face bounding box
- Mask/No Mask label
- Permission message

29 CHECK KEYBOARD
If "q" key is pressed, exit the loop.

30 CLOSE EVERYTHING
Close OpenCV windows.
Release the webcam.

BLOB = Preprocessed image for SSD model
ROI = Region of Interest (cropped face)
SSD = Single Shot MultiBox Detector

MASK (Allowed)
NO MASK (Not Allowed)

Models Used
faceNet → OpenCV SSD Face Detector
maskNet → Trained MobileNetV2 (FACEMASK_Detector.h5)